

LAPORAN TUGAS PRAKTIKUM STRUCTUR DATA

PEKAN 5

SINGLE LINKED LIST



Disusun Oleh :

UMAR ABDUL AZIZ

NIM 2511532012

DOSEN PENGAMPU Wahyudi Dr.S.T.M.T

DEPARTMENT INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG 7 MEI 2026

KATA PENGHANTAR

Puji syukur kehadiran Allah SWT yang telah memberikan rahmat, taufik, dan hidayahNya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur Data dengan baik dan tepat waktu. Laporan ini disusun untuk memenuhi salah satu tugas praktikum pada pertemuan kedua dengan judul “Single Linked List”. Melalui penyusunan laporan tugas ini, penulis berharap dapat memberikan gambaran mengenai arraylist dengan java.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan maupun pemahaman di masa yang akan datang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu serta semua pihak yang telah memberikan bimbingan, arahan, dan bantuan hingga laporan ini dapat terselesaikan. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 7 Mei 2026

Penulis

DAFTAR ISI dsadwadsad

KATA PENGHANTAR.....	2
DAFTAR ISI dsadwadsad.....	3
BAB I PENDAHULUAN.....	4
1.1 Latar Belakang.....	4
1.2 Tujuan Praktikum	4
1.3 Persyaratan Praktikum	4
BAB II TUGAS PRAKTIKUM.....	5
2.1 Deskripsi Tugas.....	5
2.2 Kode Program	5
2.3 Penjelasan isi Program.....	9
2.4. Hasil Program	10
BAB III KESIMPULAN.....	13
3.1 Kesimpulan	13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi memberikan banyak kemudahan dalam berbagai bidang, termasuk pada pelayanan rumah sakit. Salah satu permasalahan yang sering terjadi adalah pengelolaan antrian pasien yang masih dilakukan secara manual sehingga dapat menyebabkan ketidakteraturan data dan memperlambat proses pelayanan. Oleh karena itu, diperlukan sebuah sistem sederhana yang dapat membantu proses pengelolaan antrian pasien agar lebih teratur dan efisien. Dalam pemrograman, struktur data Single Linked List dapat digunakan untuk mengelola data antrian secara dinamis karena setiap data disimpan dalam node yang saling terhubung menggunakan pointer atau referensi.

Pada tugas ini, konsep Single Linked List diterapkan dalam simulasi antrian rumah sakit dengan metode FIFO (First In First Out), yaitu pasien yang datang terlebih dahulu akan dipanggil lebih dahulu. Setiap pasien direpresentasikan sebagai node yang menyimpan nama pasien, penyakit, nomor antrian, dan pointer ke node berikutnya. Program yang dibuat memiliki fitur menambahkan pasien, memanggil pasien, menampilkan antrian, mencari pasien, dan mengecek status antrian. Dengan adanya program ini, mahasiswa diharapkan dapat memahami penerapan struktur data Single Linked List menggunakan bahasa pemrograman Java dalam menyelesaikan permasalahan antrian di kehidupan nyata.

1.2 Tujuan Praktikum

- Memahami konsep dan penerapan struktur data Single Linked List dalam bahasa pemrograman Java
- Mampu mengimplementasikan metode FIFO dalam bahasa pemrograman.
- Mampu menganalisis cara kerja single linked list dalam pengolahan data.

1.3 Persyaratan Praktikum

- Telah mengikuti perkuliahan teori Struktur data sebagai dasar pemahaman.
- Membawa perlengkapan yang diperlukan, antara lain laptop atau computer yang sudah terpasang Java Development Kit (JDK) dan Integrated Development Environment (IDE) yang direkomendasikan.
- Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
- Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
- Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

BAB II

TUGAS PRAKTIKUM

2.1 Deskripsi Tugas

Pada tugas ini, mahasiswa diminta untuk mengimplementasikan konsep Single Linked List yang bekerja secara dinamis menggunakan pointer (referensi antar node). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian pasien, di mana setiap pasien direpresentasikan sebagai sebuah node yang memiliki pointer ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (FIFO — First In, First Out).

2.2 Kode Program

- Kelas Pasien

```
package TugasPekan5_2511532012;

public class Pasien_2511532012 {
    String nama;
    String penyakit;
    int antrian ;
    Pasien_2511532012 next;

    //CONSTRUCTOR
    public Pasien_2511532012(String nama, String penyakit, int antrian) {
        this.nama = nama;
        this.penakit = penyakit;
        this.antrian = antrian;
        this.next = null;
    }

    //GETTER
    public String getnama() {
        return nama;
    }
    public String getpenyakit() {
        return penyakit;
    }
    public int getantrian() {
        return antrian;
    }
    public Pasien_2511532012 getNext() {
        return next;
    }

    //SETER
    public void setnama(String nama) {
        this.nama = nama;
    }
}
```

```

• public void setpenyakit(String penyakit) {
•     this.penyakit = penyakit;
• }
• public void setantrian(int antrian) {
•     this.antrian = antrian;
• }
• public void setnext(Pasien_2511532012 next) {
•     this.next = next;
• }
• }

```

- **Kelas Drive**

```

• package TugasPekan5_2511532012;
•
• import java.util.Scanner;
•
• public class RumahSakit_2511532012 {
•     static int noantrian = 1;
•     static Pasien_2511532012 head = null;
•     // MENU
•     public static void tampilkanMenu() {
•         System.out.println("\n== Antrian Rumah Sakit NIM: 2511532012 ==");
•         System.out.println("1. Daftarkan Pasien");
•         System.out.println("2. Panggil Pasien");
•         System.out.println("3. Tampilkan Antrian");
•         System.out.println("4. Cari Pasien");
•         System.out.println("5. Cek Status Antrian");
•         System.out.println("6. Keluar");
•     }
•     // MASUKKAN PASIEN
•     public static Pasien_2511532012 insertattail(Pasien_2511532012 head,
String nama, String penyakit) {
•         Pasien_2511532012 newP =
•             new Pasien_2511532012(nama, penyakit, noantrian);
•         System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " +
noantrian);
•         noantrian++;
•         if (head == null) {
•             return newP;
•         }
•         Pasien_2511532012 last = head;
•         while (last.getNext() != null) {
•             last = last.getNext();
•         }
•         last.setnext(newP);
•         return head;
•     }
•     // PANGGIL EPESIEN
•     public static Pasien_2511532012 deletehead(Pasien_2511532012 head) {
•         if (head == null) {
•             System.out.println("Antrian kosong");
•             return null;
•         }
•         System.out.println("Pasien " + head.getnama() + " dipanggil");

```

```

    return head.getNext();
}
// DISPLAY
public static void display(Pasien_2511532012 head) {
    if (head == null) {
        System.out.println("Tidak ada Antrian");
        return;
    }
    Pasien_2511532012 current = head;
    System.out.println("\n=== DAFTAR ANTRIAN ===");
    while (current != null) {
        System.out.println("Nama      : " + current.getnama());
        System.out.println("Penyakit : " + current.getpenyakit());
        System.out.println("No Antrian : " + current.getantrian());
        System.out.println("=====");
        current = current.getNext();
    }
}
// SEARCH
public static void Search(Pasien_2511532012 head, String nama) {
    if (head == null) {
        System.out.println("Tidak ada Antrian");
        return;
    }
    while (head != null) {
        if (head.getnama().equalsIgnoreCase(nama)) {
            System.out.println("Pasien ditemukan");
            System.out.println("No Antrian : " + head.getantrian());
            System.out.println("Nama      : " + head.getnama());
            System.out.println("Penyakit  : " + head.getpenyakit());
            return;
        }
        head = head.getNext();
    }
    System.out.println("Pasien tidak ditemukan");
}
// STATUS ANTRIAN
public static void statusAntrian(Pasien_2511532012 head, String nama) {
    if (head == null) {
        System.out.println("Antrian kosong");
        return;
    }
    int posisi = 1;
    while (head != null) {
        if (head.getnama().equalsIgnoreCase(nama)) {
            System.out.println("Pasien ditemukan");
            System.out.println("Nama      : " + head.getnama());
            System.out.println("No Antrian : " + head.getantrian());
            System.out.println("Posisi    : " + posisi);
            System.out.println("Status    : Masih menunggu");
            return;
        }
        head = head.getNext();
        posisi++;
    }
}

```

```

        System.out.println("Status: Pasien tidak ada / sudah dipanggil");
    }
    // MAIN
    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        int choice;

        do {
            tampilkanMenu();
            System.out.print("Pilihan: ");
            choice = scanner.nextInt();
            scanner.nextLine();

            switch (choice) {

                case 1:
                    System.out.print("Masukkan Nama Pasien : ");
                    String nama = scanner.nextLine();

                    System.out.print("Masukkan Keluhan      : ");
                    String penyakit = scanner.nextLine();

                    head = insertattail(head, nama, penyakit);
                    break;

                case 2:
                    head = deletehead(head);
                    break;

                case 3:
                    display(head);
                    break;

                case 4:
                    System.out.print("Masukkan Nama Pasien : ");
                    String cari = scanner.nextLine();
                    Search(head, cari);
                    break;

                case 5:
                    System.out.print("Masukkan Nama Pasien : ");
                    String status = scanner.nextLine();
                    statusAntrian(head, status);
                    break;

                case 6:
                    System.out.println("Keluar dari Program");
                    break;

                default:
                    System.out.println("Pilihan Tidak Valid");
            }

        } while (choice != 6);
    }
}

```



```

•
• scanner.close();
• }
• }

```

2.3 Penjelasan isi Program

- **Kelas Pasien**

Class Pasien_2511532012 digunakan sebagai representasi node pada struktur data Single Linked List dalam program simulasi antrian rumah sakit. Setiap node menyimpan data pasien berupa nama pasien, penyakit atau keluhan pasien, nomor antrian, serta pointer atau referensi menuju node berikutnya. Variabel next digunakan untuk menghubungkan satu node dengan node lainnya sehingga membentuk sebuah linked list yang dinamis.

Constructor pada class ini berfungsi untuk menginisialisasi data pasien ketika objek dibuat. Pada constructor, nilai nama pasien, penyakit, dan nomor antrian diisi sesuai dengan parameter yang diberikan. Selain itu, variabel next diatur bernilai null karena node baru belum terhubung dengan node lain pada linked list.

Method getter pada class digunakan untuk mengambil atau membaca nilai atribut yang terdapat pada objek pasien. Method seperti getnama(), getpenyakit(), dan getantrian() digunakan untuk mengambil data nama pasien, penyakit pasien, dan nomor antrian. Sedangkan method getNext() digunakan untuk mengambil referensi node berikutnya pada linked list.

Method setter digunakan untuk mengubah atau memperbarui nilai atribut pada objek pasien. Method setnama(), setpenyakit(), dan setantrian() digunakan untuk mengubah data pasien, sedangkan method setnext() digunakan untuk menghubungkan node satu dengan node lainnya. Dengan adanya method getter dan setter, proses pengolahan data pasien menjadi lebih terstruktur dan mudah dikelola.

- **.Kelas Drive**

Class RumahSakit_2511532012 merupakan class utama yang digunakan untuk menjalankan program simulasi antrian rumah sakit menggunakan struktur data Single Linked List. Pada class ini terdapat variabel static noantrian yang digunakan untuk memberikan nomor antrian secara otomatis kepada setiap pasien yang mendaftar. Selain itu, terdapat variabel head yang berfungsi sebagai penunjuk node pertama pada linked list. Jika linked list masih kosong, maka nilai head adalah null.

Method tampilkanMenu() digunakan untuk menampilkan daftar menu program kepada pengguna. Menu yang tersedia meliputi pendaftaran pasien, pemanggilan pasien, menampilkan antrian, mencari pasien, mengecek status antrian, dan keluar dari program. Method ini membantu pengguna untuk memilih operasi yang ingin dijalankan pada sistem antrian rumah sakit.

Method insertattail() digunakan untuk menambahkan pasien baru ke bagian akhir linked list. Pada method ini dibuat node baru menggunakan constructor class Pasien_2511532012 yang berisi nama pasien, penyakit, dan nomor antrian otomatis.

Jika linked list masih kosong, maka node baru langsung menjadi head. Namun jika linked list sudah memiliki isi, program akan melakukan traversal hingga menemukan node terakhir kemudian menghubungkannya dengan node baru menggunakan pointer next.

Method deletehead() digunakan untuk memanggil atau menghapus pasien pertama dalam antrian. Method ini menerapkan konsep FIFO (First In First Out), yaitu pasien yang datang lebih dahulu akan dipanggil lebih dahulu. Jika antrian kosong maka program akan menampilkan pesan bahwa tidak ada pasien dalam antrian. Namun jika terdapat pasien, maka node pertama akan dihapus dengan memindahkan head ke node berikutnya.

Method display() digunakan untuk menampilkan seluruh data pasien yang terdapat dalam antrian. Program melakukan traversal dari node pertama hingga node terakhir menggunakan variabel bantu current. Pada setiap node, program menampilkan nama pasien, penyakit, dan nomor antrian sehingga seluruh isi linked list dapat dilihat oleh pengguna.

Method Search() digunakan untuk mencari data pasien berdasarkan nama pasien. Program akan melakukan traversal pada linked list dan membandingkan nama pasien yang dicari dengan data pada setiap node menggunakan equalsIgnoreCase(). Jika data ditemukan, maka informasi pasien akan ditampilkan. Namun jika traversal selesai dan data tidak ditemukan, maka program akan menampilkan pesan bahwa pasien tidak ditemukan.

Method statusAntrian() digunakan untuk mengecek status pasien dalam antrian rumah sakit. Method ini tidak hanya mencari pasien, tetapi juga menentukan posisi pasien dalam antrian. Program menggunakan variabel posisi untuk menghitung urutan pasien selama proses traversal berlangsung. Jika pasien ditemukan, maka program akan menampilkan nama pasien, nomor antrian, posisi antrian, dan status bahwa pasien masih menunggu giliran.

Method main() merupakan method utama yang digunakan untuk menjalankan seluruh program. Pada method ini digunakan Scanner untuk menerima input dari pengguna. Program dijalankan menggunakan perulangan do-while sehingga menu akan terus tampil sampai pengguna memilih menu keluar. Pemilihan menu dilakukan menggunakan struktur switch-case untuk menjalankan fungsi sesuai pilihan pengguna. Dengan adanya method ini, seluruh fitur pada sistem antrian rumah sakit dapat dijalankan secara terstruktur dan interaktif

2.4. Hasil Program

- Menu 1

```
• === Antrian Rumah Sakit NIM: 2511532012 ===
• 1. Daftarkan Pasien
• 2. Panggil Pasien
• 3. Tampilkan Antrian
• 4. Cari Pasien
• 5. Cek Status Antrian
• 6. Keluar
```

- Pilihan: 1
- Masukkan Nama Pasien : Jun
- Masukkan Keluhan : Deman
- Pasien berhasil didaftarkan! Nomor Antrian: 1

- Menu 2

- === Antrian Rumah Sakit NIM: 2511532012 ===
- 1. Daftarkan Pasien
- 2. Panggil Pasien
- 3. Tampilkan Antrian
- 4. Cari Pasien
- 5. Cek Status Antrian
- 6. Keluar
- Pilihan: 2
- Pasien Jun dipanggil

- Menu 3

- === Antrian Rumah Sakit NIM: 2511532012 ===
- 1. Daftarkan Pasien
- 2. Panggil Pasien
- 3. Tampilkan Antrian
- 4. Cari Pasien
- 5. Cek Status Antrian
- 6. Keluar
- Pilihan: 3
-
- === DAFTAR ANTRIAN ===
- Nama : Jun
- Penyakit : Deman
- No Antrian : 1
- =====
- Nama : Huan
- Penyakit : flu
- No Antrian : 2
- =====

- Menu 4

- === Antrian Rumah Sakit NIM: 2511532012 ===
- 1. Daftarkan Pasien
- 2. Panggil Pasien
- 3. Tampilkan Antrian
- 4. Cari Pasien
- 5. Cek Status Antrian
- 6. Keluar
- Pilihan: 4
- Masukkan Nama Pasien : Huan
- Pasien ditemukan
- No Antrian : 2
- Nama : Huan
- Penyakit : flu
-

- Menu 5

- === Antrian Rumah Sakit NIM: 2511532012 ===
- 1. Daftarkan Pasien
- 2. Panggil Pasien

- 3. Tampilkan Antrian
- 4. Cari Pasien
- 5. Cek Status Antrian
- 6. Keluar
- Pilihan: 5
- Masukkan Nama Pasien : Jun
- Status: Pasien tidak ada / sudah dipanggil

- Menu 6

- === Antrian Rumah Sakit NIM: 2511532012 ===
- 1. Daftarkan Pasien
- 2. Panggil Pasien
- 3. Tampilkan Antrian
- 4. Cari Pasien
- 5. Cek Status Antrian
- 6. Keluar
- Pilihan: 6
- Keluar dari Program

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan tugas yang telah dibuat, dapat disimpulkan bahwa struktur data Single Linked List dapat digunakan untuk mengimplementasikan sistem antrian rumah sakit secara dinamis dan terstruktur. Setiap pasien direpresentasikan sebagai node yang saling terhubung melalui pointer sehingga memudahkan proses pengelolaan data antrian.

Program yang dibuat telah berhasil menerapkan konsep FIFO (First In First Out), yaitu pasien yang datang terlebih dahulu akan dipanggil terlebih dahulu. Fitur-fitur seperti menambahkan pasien, memanggil pasien, menampilkan antrian, mencari pasien, dan mengecek status antrian dapat berjalan dengan baik menggunakan operasi pada Single Linked List.

Melalui tugas ini, pemahaman mengenai penggunaan pointer, traversal, insert, delete, dan search pada struktur data Single Linked List menjadi lebih baik. Selain itu, tugas ini juga membantu mahasiswa memahami penerapan struktur data dalam menyelesaikan permasalahan nyata, khususnya pada sistem antrian rumah sakit menggunakan bahasa pemrograman Java.